MLOps Assignment 2

Hugging Face Fine-Tuning, Experiment Tracking & Model Deployment

PGD AI Program | IIT Jodhpur

Aishwarya Mishra
Roll No: G25AIT2137

1. Introduction

This report documents an end-to-end MLOps workflow that fine-tunes a pre-trained transformer model for a multi-class text-classification task. The task is to predict the book genre of a Goodreads review across eight classes: poetry, children, comics_graphic, fantasy_paranormal, history_biography, mystery_thriller_crime, romance, and young_adult. The dataset is sourced from the UCSD Goodreads reviews collection. The focus of this assignment is not raw model accuracy but the operational pipeline around the model: training on a managed GPU environment (Kaggle), tracking experiments with Weights & Biases (W&B), and publishing the trained artifact to the Hugging Face Hub so it can be reused by anyone.

2. Model Selection Rationale

The pre-trained model selected for fine-tuning is distilbert-base-cased, distributed by Hugging Face. DistilBERT is a knowledge-distilled version of BERT-base that retains roughly 97% of BERT's language-understanding performance on the GLUE benchmark while being approximately 40% smaller (66M vs 110M parameters) and about 60% faster at inference. The cased variant preserves capitalisation, which is helpful for review text where proper nouns (character names, titles) carry useful signal. For this assignment three constraints made DistilBERT the right choice: (i) free-tier Kaggle GPU sessions have limited memory and runtime, so a lighter model lets training finish well within the 9-hour session limit; (ii) the dataset (6,400 training reviews) is small, so the additional capacity of larger BERT variants would offer minimal benefit and risk overfitting; and (iii) DistilBERT's smaller footprint makes the resulting Hugging Face repository quicker to pull and easier to use downstream. The model card on Hugging Face confirms broad community usage and reproducible benchmarks, which made it a safe baseline choice.

3. Kaggle Training Setup

Training was performed entirely inside a Kaggle Notebook to leverage Kaggle's free GPU quota. The notebook is configured as follows:

- Accelerator: GPU T4 x2 (selected via Settings → Accelerator).
- Internet: enabled, required for downloading the UCSD dataset and for pushing artifacts to W&B and Hugging Face.
- Add-ons → Secrets: two secrets were registered and attached to the notebook - WANDB_API_KEY (from wandb.ai/settings) and HF_TOKEN (a write-scope token from huggingface.co/settings/tokens). Secrets are read at runtime through kaggle_secrets.UserSecretsClient and exported to environment variables, so credentials are never hardcoded.
- Reproducibility: random seeds for Python, NumPy and PyTorch were fixed at 42.

Hyperparameters used for the fine-tuning run are summarised below.

Hyperparameter	Value
Model	distilbert-base-cased

Epochs	3
Train batch size	16
Eval batch size	32
Learning rate	3e-5
Warmup steps	100
Weight decay	0.01
Max sequence length	512
Samples per genre	1000 (800 train / 200 test)
Total train / test	6400 / 1600
Hardware	Kaggle GPU (T4 x2)

4. Training Summary & W&B Tracking

Fine-tuning uses the Hugging Face Trainer API. report_to="wandb" in TrainingArguments delegates all metric and system logging to W&B automatically, which means every loss step, evaluation epoch, learning-rate change and GPU-utilisation sample is captured without additional code. wandb.init also logs the full hyperparameter config so each run is fully reproducible. The compute_metrics function returns accuracy and weighted F1 after every evaluation epoch; load_best_model_at_end=True with metric_for_best_model="f1" ensures the best checkpoint is kept regardless of which epoch produced it. The W&B dashboard provided live visibility into training loss, validation loss, accuracy, F1, learning-rate schedule and GPU memory usage.

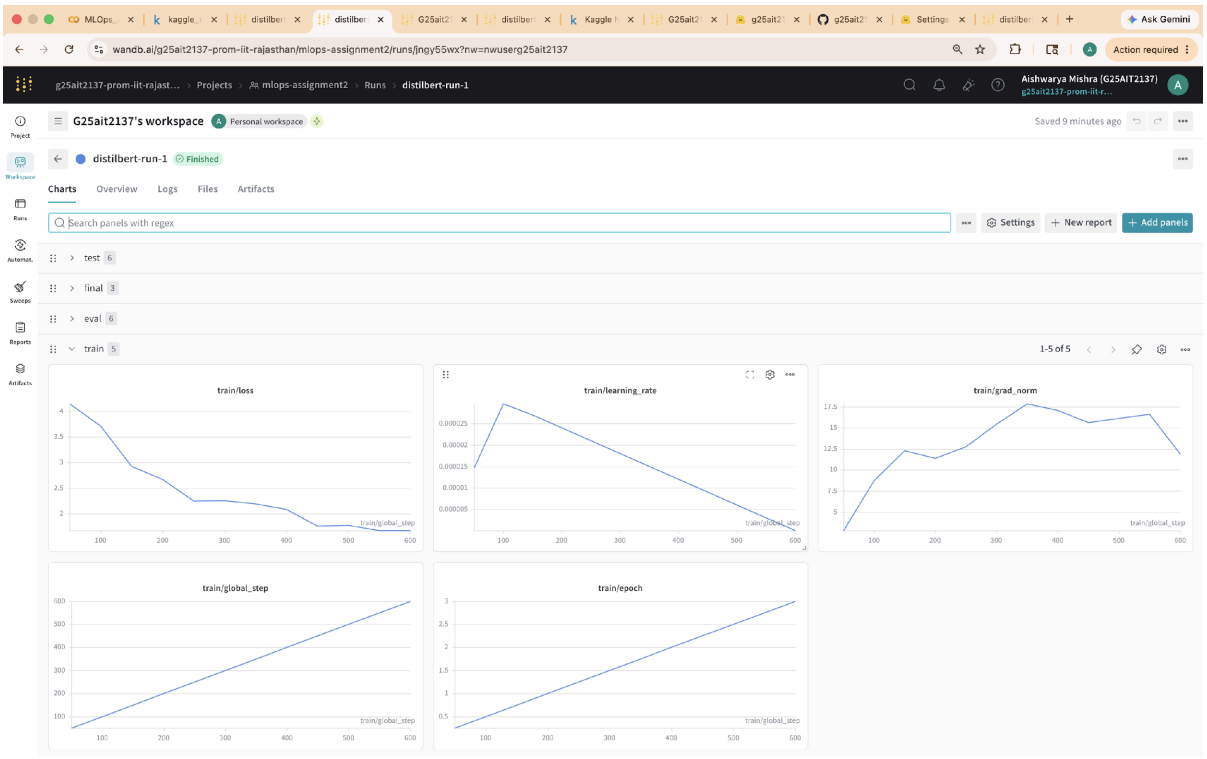

Figure 1. W&B dashboard for run distilbert-run-1 (project: mlops-assignment2).

5. Evaluation Results

After training, trainer.evaluate() was run on the held-out test set (1,600 reviews, 200 per genre). The final loss, accuracy and weighted F1 are explicitly logged to W&B under the final/* namespace. A full per-class classification report is saved as eval_report.json and uploaded as a versioned W&B Artifact (type="evaluation"), so the exact evaluation snapshot is retrievable from the run history.

Metric	Score
Accuracy	0.6006
F1 Score (weighted)	0.5972
Eval Loss	2.2605

On the test set of 1,600 reviews, the fine-tuned DistilBERT achieved 60.1% accuracy and a weighted F1 of 0.597 across the eight genres - well above the random baseline of 12.5% for an eight-class problem. The per-class numbers (saved in eval_report.json) tell a more interesting story: the model is strongest on comics_graphic (F1 = 0.80) and poetry (F1 = 0.78), where the language is stylistically distinctive, and weakest on young_adult (F1 = 0.34) and fantasy_paranormal (F1 = 0.47), which overlap heavily with other genres (especially romance and children's fiction) in the language reviewers use. This pattern is consistent with the fact that genre boundaries on Goodreads are semantically fuzzy rather than crisp categories. Improving the weaker classes would likely need more training data or a multi-label formulation rather than a bigger model, which aligns with the assignment's emphasis on workflow rather than raw performance.

6. Publishing to Hugging Face Hub

Once training finished, the fine-tuned model weights and the matching tokenizer were pushed to the Hugging Face Hub under g25ait2137/distilbert-goodreads-genres. The repository is set to public so it can be loaded with a single from_pretrained call by any downstream user. The Hub URL is also written into wandb.run.summary["huggingface_model"], linking the trained artifact back to the W&B run that produced it.

7. Challenges & Learnings

- Working inside a managed notebook environment (Kaggle) is different from a local IDE: secrets must be registered through the Kaggle UI, and the Internet toggle has to be on for any outbound calls to W&B or Hugging Face - turning it off silently breaks model pushes.
- Streaming a gzipped JSON dataset over HTTP is fast but fragile; persisting the loaded reviews to a pickle file early in the notebook avoided having to re-download on every session restart.
- Wiring W&B into Trainer is essentially a single argument (report_to="wandb"), but the value of that one flag is significant - it turns an opaque training run into a fully observable experiment with system metrics, hyperparameters and artifacts all linked in one place.
- Using load_best_model_at_end together with metric_for_best_model="f1" was important: with a small dataset, the best epoch is not always the last one, and saving the best checkpoint avoided shipping a suboptimal model to the Hub.
- If repeated on a different task, I would also log the confusion matrix and per-class F1 directly as W&B custom charts rather than only as a JSON artifact, so reviewers can see model behaviour without downloading the report.

8. Submission Links

GitHub repository: https://github.com/g25ait2137-ops/mlops-assignment2
Hugging Face model: https://huggingface.co/g25ait2137/distilbert-goodreads-genres
W&B dashboard: https://wandb.ai/g25ait2137-prom-iit-rajasthan/mlops-assignment2
Kaggle notebook: https://www.kaggle.com/code/aishwaryamishra8/kaggle-notebook-ipynb